When dependency backpressure helps - and when a ready queue is better

SwarmCircuit: replayable evaluation and reviewed cooperative containment

Author and affiliation metadata require confirmation before submission.
Prepared for the Apart Research Swarm Alignment Hackathon; not submitted or endorsed.

Abstract

Multi-agent workflows can amplify requests while several actors wait for a shared approval. We evaluate whether dependency-aware backpressure adds value beyond request deduplication, retry limits, rate limits and scheduling. A bounded six-task simulator separates tool and host truth from delivered observations. We compare eight policies on nine scenarios with thirty matched seeds each, yielding 2,160 completed trials. In the repeated-gate scenario, backpressure reduces mean coordination requests from 62.73 under exact-key deduplication to 17.27, with a paired difference of -45.47 requests (95% bootstrap interval -49.67 to -40.93). A dependency-ready queue requires only 6.00 requests, providing a stronger comparison than reactive controls. Normal and legitimate-wait scenarios incur no backpressure holds. Fault injection exposes important limits: an omitted approval update prevents completion of gated work, and an incorrect dependency edge blocks otherwise independent work under both graph-dependent policies. A separate browser-driven SDK demonstration obtains four matching runtime acknowledgements while an independent branch completes. Two live-model pilot attempts fail at the provider and provide no comparative model result. These findings support a narrow claim about request suppression in this synthetic environment, not production safety or adversarial containment. All implementation and evaluation reported here were prepared before the hackathon period.

Abstract: 195 words. Evidence: synthetic trials plus separate cooperative SDK checks.

1. Introduction

Repeated requests and shared waits can create coordination overhead even when every business-action guard remains intact. The practical question is whether pausing a dependency branch offers measurable benefit beyond simpler prevention and suppression mechanisms.

Our contributions are a replayable policy comparison with independent outcome truth; explicit counterexamples involving graph and observation errors; and an operator-reviewed bridge from a dependency preview to physically parked SDK boundaries. The ready-queue comparison is retained even where it is superior. No claim of a new general multi-agent safety guarantee is made.

2. Methods

2.1 Environment and matched inputs

The DAG contains plan (initially complete), access, research, validate, report and public. Access depends on plan; research and validate depend on access; report depends on both; public depends only on plan. The latter five nodes are goals. The gate guards access and its dependent branch. No protected action can begin without authoritative approval and completed true prerequisites.

Thirty seeds (0-29), sixty logical ticks and eight arms are fixed in the manifest. Each scenario/seed shares world inputs across arms. Seeded gate delay is 18-27 ticks; legitimate wait is 8; normal is immediate. Task durations and proposal schedules vary by seed. Reactive actors in normal/wait cases do not repeatedly request an unchanged blocked state. Gate observations reset retry counters and backoff and clear dedup caches; prerequisite completion invalidates dependent cached blocks. Request counts include policy-admitted proposals, excluding suppressed actor wakeups.

Policy	Definition
B0	Safety guards only
B1	Three attempts per agent per observed gate version
B2	Global two-tool-request-per-tick cap
B3	Exact-key deduplication
B4	B3 plus reverse dependency closure parking
B4_no_graph	B4 root-only ablation
B5	Exponential retry backoff, capped at 16 ticks
A_ready_queue	Dependency-ready queue plus deduplication

2.2 Faults and observability

Nine scenarios cover normal execution, legitimate waiting, retry cascades, a pending gate, a denied gate, seven-tick delayed observations, an omitted approval update, a host unavailable at ticks 5-25, and an incorrect graph edge. The graph-error case falsely makes public depend on access; public proposals begin at tick 12 in all arms to exercise the faulty closure. This is a designed counterexample, not a prevalence estimate.

Policies consume delivered observations; the reducer consumes separate simulated tool/host truth. B4 parks the observed reverse dependency closure after three pending demands and releases it after a delivered gate update. The closure function is shared with the production detector. Gate safety is identical across arms and is enforced by the simulator, independently of the policy.

2.3 Production policy differs from the model

The modeled B4 policy acts automatically. Production application requires an operator reason, target preview, exact-preview hash, current checkpoint ownership, fencing epoch and matching runtime ACK. Production resume remains an explicit review. Logical-tick effects therefore do not estimate operator response time, deployment latency or production policy performance.

3. Results: request overhead

All 2,160 planned synthetic trials completed; none failed or were missing. Offline verification reproduced every completed trial's artifact hash, outcome metrics and CSV measurements without provider calls. Per-scenario/per-arm estimates use n=30 seeds.

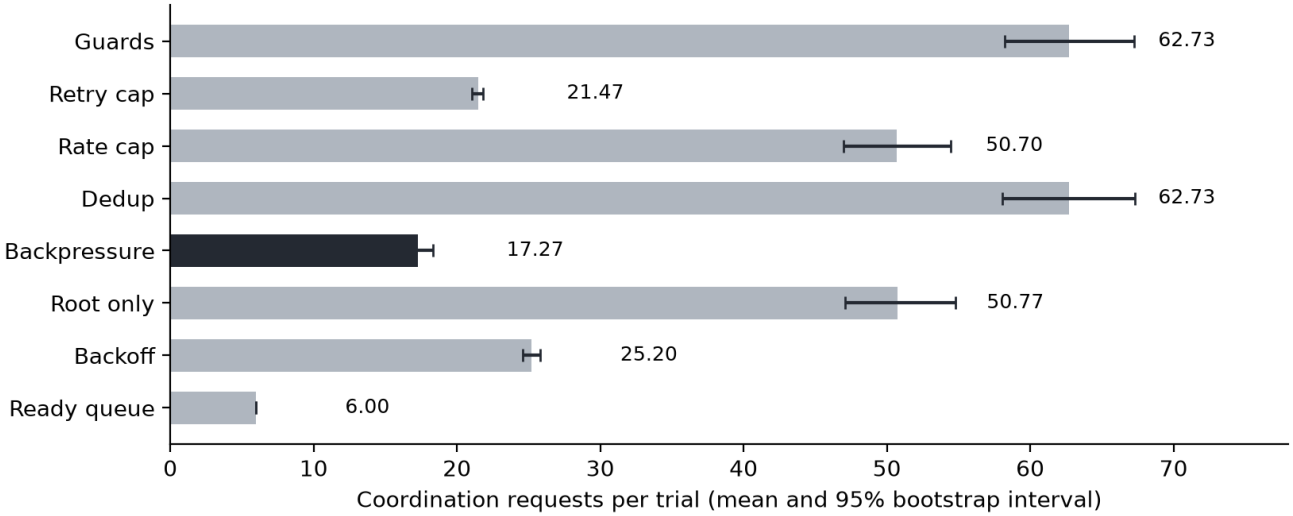


Figure 1. Repeated shared-gate requests. Bars are seed means; whiskers are 95% percentile-bootstrap intervals from 1,000 seed resamples. Every arm uses identical safety guards. Lower request counts do not alone establish successful completion.

Policy	Requests	Guard checks	Permitted completion
B0	62.73	62.73	100.0%
B1	21.47	21.47	84.0%
B2	50.70	50.70	100.0%
B3	62.73	12.30	100.0%
B4	17.27	11.80	100.0%
B4_no_graph	50.77	12.30	100.0%
B5	25.20	25.20	100.0%
A_ready_queue	6.00	6.00	100.0%

Table 1. Repeated-gate scenario means. B1 lowers completion to 84.0% (95% interval 78.7%-88.7%), illustrating why a request reduction needs a completion measure. B4, B3, B5 and the ready queue complete all permitted goals in this scenario.

Relative to B3, B4 changes requests by -45.47 per trial (paired 95% interval -49.67 to -40.93). This is a 72.5% reduction in the ratio of sample means. Guard checks change only -0.50 (interval -0.77 to -0.30): deduplication already removes most redundant checks. Both deliver one notification on average. The ready queue uses 6.00 requests and 6.00 guard checks.

Normal execution uses 5.00 requests under every arm. In legitimate waiting, B4 matches B3 at 6.87; the ready queue uses 6.00. B4 records zero oracle-ready held task-ticks in both cases. These scenarios do not demonstrate a benefit from intervention.

4. Results: counterexamples

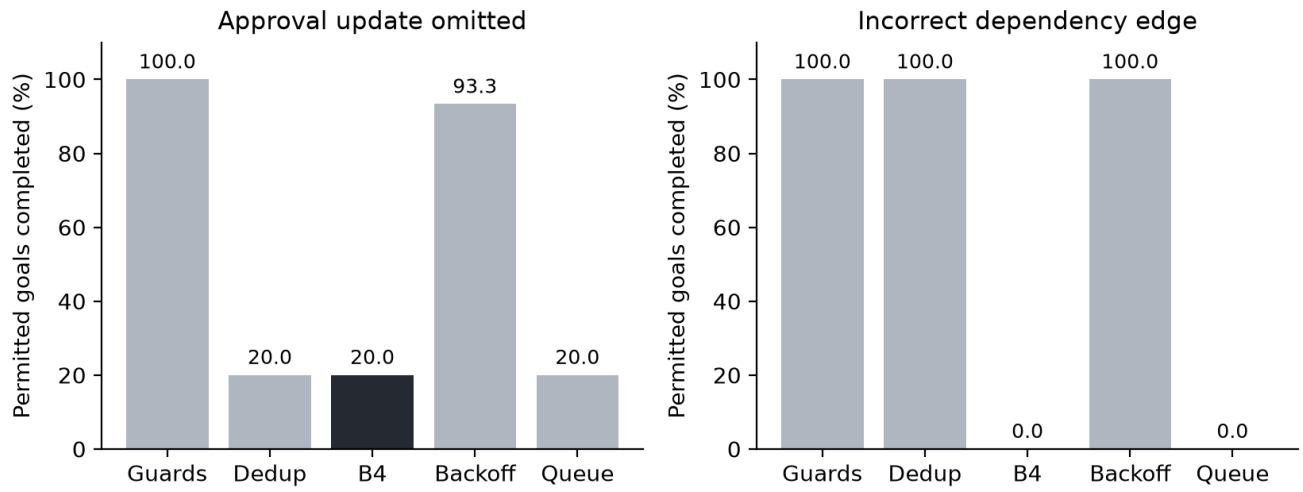


Figure 2. Fault scenarios, n=30 per arm. Denominators depend on authoritative gate state: five goals after approval; only public while the gate remains pending. Thus 20% with a lost update means public alone completes, whereas 0% with the incorrect graph means public also fails to complete. All bars use simulator truth.

Scenario	B3	B4	Ready queue
Pending gate: requests	141.33	7.03	2.00
Denied gate: requests	141.33	95.87	24.73
Delayed telemetry: completion	100%	100%	100%
Lost update: completion	20%	20%	20%
Wrong graph: independent completion	100%	0%	0%
Wrong graph: independent latency	13.97	60 (censored)	60 (censored)

Table 2. Selected fault outcomes. Full outcomes and uncertainty for every metric/scenario/arm are retained in results.json and metrics.csv. There is no universal winning arm.

4.1 Missing observations create a liveness tradeoff

When the approval update is omitted, B4 remains parked after truth changes to approved: permitted completion is 20% and oracle-ready held time averages 37.53 task-ticks (95% interval 36.53-38.53). B3 and the queue also achieve only 20%, because their observation-dependent suppression does not refresh. Guard-only actors complete 100%; backoff completes 93.3%. A ready queue needs a reconciliation mechanism when its observations may be incomplete.

4.2 A graph error suppresses independent work

The incorrect edge causes B4 and the ready queue to prevent public from completing in every seed. B4 records 57.03 oracle-ready held task-ticks on average; root-only parking preserves public completion. The queue records zero false-hold ticks because it suppresses scheduling rather than parks a host: the completion outcome is essential to expose the same loss of liveness.

Delayed telemetry preserves B4 completion but adds seven oracle-ready held task-ticks. Under temporary disconnection, B4 eventually completes all permitted goals in this bounded case. Neither result guarantees recovery under arbitrary delay or failure.

5. Engineering demonstration and evidence

5.1 Reviewed group containment

A separate live local SDK run creates four reversible guarded boundaries sharing one pending mock approval. BrowserAct opens Approvals, records a reason, previews four eligible targets and two outside-branch nodes, and submits the exact preview hash. Each runtime records a matching applied ACK at epoch 1. Public then completes while the approval remains pending. Reloading the browser reconciles the same durable policy request and its four ACKs.

Observed engineering check	Outcome
Run ID	ff817850-735f-4485-b9be-4dbc9a2fd850
Reviewed / matching applied holds	4 / 4
Independent branch completed	Yes
Protected mock effects	0
Approval state	Pending
Telemetry degraded	No

Table 3. One cooperative SDK check, not thirty replicated runtime trials. The bounded host exits after the ACK check; historical ACKs do not establish that a departed process is currently alive. No payment, real approval resolution or external business effect is exercised.

5.2 Live-model pilot: unavailable

DeepSeek-V4-Flash was requested through api.hcnsec.cn using its OpenAI-compatible chat-completions interface. One connectivity smoke call succeeded outside the study. Two study attempts consumed one and two request attempts respectively before provider_unavailable. All eight policy trials in each attempt are marked failed with no numeric outcomes. One successful action response in the second attempt is retained only as a transcript; a partial workload is not presented as a completed benchmark.

The planned pilot uses five LLM actors, a fixed planning node, temperature 0.2, maximum 512 output tokens per call, a bounded total-call budget and synthetic metadata only. It records attempt/wait proposals under guard-only feedback, then replays the same tape across policies. Even a successful pilot would test open-loop workload replay, not adaptive counterfactual behavior. Failed calls have unknown token usage; no dollar savings are estimated.

5.3 Validation and provenance

The implementation passes 149 backend tests, Ruff and mypy, twelve Studio unit cases, strict TypeScript, a production Next.js build, and five live HTTP/WebSocket gateway cases. Trial evidence seals truth, observations, proposal tape, protocol and source fingerprints. CSV exports include denominators and failures. Research projections are labeled Observe/simulated and separate from ordinary workflows.

These engineering checks establish specific implemented behaviors and invariants. They do not validate an untrusted host, adversarial agents, a real payment service or a remotely deployed multi-user authorization system.

6. Discussion and limitations

The strongest supported claim is conditional: given this small correct graph and retrying workload, dependency backpressure reduces requests beyond deduplication. An observation-ready queue prevents still more requests. The likely practical role for reactive containment is in externally scheduled or partially integrated workflows where central scheduling cannot prevent demand; that role remains a hypothesis requiring direct evaluation.

The study has six tasks, synthetic actor rules, a sixty-tick horizon and an exploratory scenario set. Metrics are correlated; the many intervals are not adjusted for multiple comparisons. Matched seeds are independent experimental units; individual tool calls are not. Model-work units (1 request + 3 checks + 10 notices) are arbitrary fixed costs, not measured money or compute. Incomplete independent latency is censored at tick 60. Zero unsafe starts follows identical guards by construction, not an empirical estimate of low production risk.

6.1 Related work

Hammond et al. [1] distinguish miscoordination, conflict and collusion in multi-agent systems. Our scope is the narrow miscoordination mechanism of amplified blocked demand; we do not test collusion. Greenblatt et al. [2] evaluate control under intentional subversion. SwarmCircuit instead trusts cooperative runtime hooks and ACKs, and cannot inherit adversarial safety claims from that line of work.

6.2 Next experiment

During the eligible sprint, freeze a new baseline and preregister a closed-loop comparison with multiple completed live-model workloads. Add authoritative-state reconciliation for lost observations, graph-confidence checks, missing/extra edges and larger graphs. Measure intervention delay and operator burden on actual runtimes. Preserve the ready-queue comparison and report negative outcomes before claiming improved safety.

7. Conclusion

Backpressure reduces coordination demand in this synthetic repeated-gate workload, but does not dominate scheduling. Observation loss and graph mistakes expose liveness failures. Replayable evidence and reviewed runtime controls make these tradeoffs inspectable; they do not remove them.

Code and data

Protocol coordination-cascades-v1; synthetic experiment 0ada95ca-a10a-4835-ade4-31c088c2e031. Source fingerprint: 7be6aac8b415846ac88705179e8d801c235f9e381ca3a15b11292291a300f57a. CSV SHA-256: 27ea003676174da3e8122a8d4d699e867835d0273372f60fac11dea29b7c5ab2. These are content hashes, not Git commits. The accompanying local reproducibility bundle contains source, locks, raw trial journals, results, failed provider records and reproduction instructions. No credential or private database is included.

References

[1] Hammond et al. (2025). *Multi-Agent Risks from Advanced AI.* arXiv:2502.14143.
[2] Greenblatt, Shlegeris, Sachan and Roger (2024). *AI Control: Improving Safety Despite Intentional Subversion.* arXiv:2312.06942.
[3] Apart Research. Swarm Alignment Hackathon guidelines, consulted November 5, 2026. Report structure follows the linked official template.

Appendix A. Limitations, dual use and disclosure

A.1 Trust and control limits

Runtime control is cooperative. A malicious host can omit hooks, misreport observations, acknowledge dishonestly or perform an action outside a boundary. Holds cannot interrupt an action already beyond its checkpoint. Fencing epochs and matching ACKs prevent stale protocol acceptance within this design; they do not establish truthful external behavior. Approval and payment authorities remain separate.

Preview hashes reduce stale-review mistakes but are not an authorization substitute. Production requires an operator role, fresh scope validation, reversible owned checkpoints and explicit recovery. No group control is granted through inferred trace ancestry. The local dashboard trusts users of the local machine and is not a remote production authentication service.

A.2 Potential harms and dual use

Dependency graphs, actor state and intervention tools can aid operational oversight, but can also enable surveillance, denial of service or selective suppression. A mistaken graph can withhold legitimate work, as the counterexample demonstrates. Sensitive telemetry must be minimized and access scoped before real integrations. Automatic intervention should not be introduced from this study without liveness evaluation, operator review and an external authority design.

Only mock/synthetic tasks and amounts were used. No wallet, real settlement, private user workflow, external tool execution or unapproved communication was involved. Model requests contained synthetic task metadata. Credentials are excluded from source, transcripts, screenshots and the bundle.

A.3 LLM usage statement

Codex assisted with implementation, testing, analysis, report drafting and browser operation. The study's external model was the configured third-party DeepSeek-V4-Flash endpoint; provider model identity was not independently verified. Its failed pilot does not support an efficacy claim. Humans must review the paper, author list, affiliation and factual claims before submission. No external submission has been made.

A.4 Preparation and eligible contributions

All software, tests, browser checks, drafts and experiments in this report were prepared on or before November 5, 2026. The Swarm Alignment Hackathon runs November 6-8. These results must be disclosed as pre-existing preparation and cannot be described as new work performed during the event. At sprint start, record a real commit/tag or content inventory and separately track the event-period research contribution. GateKeep402 is separate pre-existing work; its source is not changed here.

A.5 Reproduction and failed studies

Run make research with the local backend on port 18005, or resume a matching study using --experiment-id. Verify the artifact directory with research.cli verify; this makes no model calls. Rebuilding the PDF requires ReportLab and Matplotlib, separate from the application runtime. The source manifest hashes the original research implementation and uv.lock; the bundle manifest hashes the delivered archive members.

Failed study IDs: c62bf7f9-78f0-4b3d-8ed5-996122ec36f2 and bb241eed-3584-4b64-aa15-1c2e2ea8bf54. Each contains eight failed policy trials, no completed comparative outcomes and null unknown token usage. Limits were 30 and 10 total calls respectively; observed attempted calls were 1 and 2. The successful connectivity smoke was a separate single call and is not a study sample.